

qPOTS: A Python Package for Sample-efficient Batch Constrained Multiobjective Bayesian Optimization

Kade E. Carlson¹, Ashwin Renganathan^{1,2}, and Peter E. Bachman¹

¹ Department of Aerospace Engineering, The Pennsylvania State University, United States  ² Institute for Computational and Data Sciences, The Pennsylvania State University, United States 

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: Fabian Scheipl 

Reviewers:

- [@jbussemaker](#)
- [@gialmisi](#)

Submitted: 14 June 2026

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](https://creativecommons.org/licenses/by/4.0/)).

Summary

qPOTS is a Python package for researchers and engineers who need to optimize expensive black-box systems with more than one competing objective. In such problems, each experiment, simulation, or hardware test can be costly, and the best answer is rarely a single design but instead a set of trade-off designs that form a Pareto front. Examples include aircraft design, process optimization, materials discovery, and machine learning model selection. qPOTS helps users decide which designs to evaluate next by combining Gaussian-process surrogate models, Thompson sampling, and evolutionary search over sampled Pareto fronts.

The package implements batch Pareto optimal Thompson sampling (qPOTS) for multiobjective Bayesian optimization (Renganathan & Carlson, 2025; Thompson, 1933). It provides tools for unconstrained and constrained benchmark functions, Gaussian-process model fitting and prediction, batch candidate selection via a maximin diversity rule, expected hypervolume tracking, and comparison with standard multiobjective Bayesian optimization acquisition functions. The package also includes qPOTS-Decoupled, a multitask mode for decoupled oracle evaluations: when objectives or constraints can be measured separately, the software can transfer information across correlated tasks and select a subset of outputs to query (Álvarez et al., 2012; Bonilla et al., 2007). Earlier project material called this mode qPOTS-DOE; the new name avoids confusion with the established abbreviation for design of experiments.

qPOTS is released under the GNU General Public License v3.0 and is built on widely used scientific Python libraries including PyTorch, GPyTorch, BoTorch, and pymoo (Balandat et al., 2020; Blank & Deb, 2020; Gardner et al., 2018; Paszke et al., 2019).

Statement of need

Multiobjective optimization is central to scientific and engineering design because improving one objective often degrades another. Classical evolutionary algorithms such as NSGA-II can approximate Pareto fronts accurately but require many objective evaluations (Deb et al., 2002). That cost is prohibitive when each evaluation requires a high-fidelity computational simulation, a physical experiment, or a hardware test that may take hours or days. Bayesian optimization addresses this by fitting probabilistic surrogate models to existing data and using them to select the next most informative experiments (Jones et al., 1998; Rasmussen & Williams, 2006; Shahriari et al., 2016).

Multiobjective Bayesian optimization is substantially harder than its single-objective counterpart. Acquisition functions must reason about a set-valued Pareto frontier rather than a scalar optimum, making them nonconvex, nondifferentiable, or expensive to optimize in batch settings. qPOTS addresses this by replacing direct acquisition-function maximization with posterior-sample optimization: it draws sample paths from the current surrogate model, solves

a comparatively inexpensive multiobjective problem on those samples using evolutionary search, and selects a diverse batch of candidates from the resulting sampled Pareto set (Renganathan & Carlson, 2025). This avoids the need for differentiable analytical acquisitions and naturally yields batches without additional approximation.

A further opportunity arises when the objectives and constraints of a problem can be observed independently. In aerodynamic design, for instance, drag and lift coefficients may be evaluated at multiple operating conditions that require separate solver runs; in chemical processes, different properties may require different laboratory analyses. Existing decoupled Bayesian optimization methods typically assume independent objective models or rely on known scalar oracle costs. qPOTS-Decoupled supports this setting by fitting a single multitask Gaussian process over all objectives and constraints jointly, using inter-task posterior correlation to decide whether decoupled evaluations are beneficial, and selecting an informative oracle subset via a Gaussian mutual-information criterion.

The target audience is researchers and engineers who need reproducible, sample-efficient multiobjective optimization workflows in Python. qPOTS is designed for method developers benchmarking against standard acquisition strategies, and for application researchers optimizing expensive custom black-box functions under limited evaluation budgets, including scenarios where not all objectives need to be queried at every iteration.

State of the field

BoTorch provides a general platform for Bayesian optimization and implements strong multiobjective acquisitions such as qEHVI, qNEHVI, and their logarithmic variants (Ament et al., 2023; Balandat et al., 2020; Daulton et al., 2020). Other influential methods include entropy-search approaches (PESMO, MESMO, JESMO) and lookahead methods such as the hypervolume knowledge gradient (Belakaria et al., 2019; Daulton et al., 2023; Garrido-Merchán & Hernández-Lobato, 2019; Tu et al., 2022). These are important baselines, and several are exposed in qPOTS workflows for direct comparison.

qPOTS complements rather than replaces BoTorch. BoTorch supplies Gaussian-process models, posterior sampling, and baseline acquisition functions, while qPOTS coordinates a cross-library workflow that is not represented by one BoTorch acquisition primitive: it optimizes sampled multiobjective paths with NSGA-II, selects a diverse batch from the sampled Pareto set, evaluates constrained or partially observed outputs, and records the resulting optimization history. Packaging that orchestration separately lets qPOTS retain direct interoperability with BoTorch and pymoo while providing a stable research workflow, benchmark registry, and high-level runner. Users can still access `ModelObject` and `Acquisition` directly when they need lower-level control.

The decoupled mode extends this software workflow to partially observed, correlated outputs. Its task-correlation gate and oracle-subset rule are described in the companion methods manuscript (Renganathan et al., 2026). This JOSS paper focuses on the implementation and user interfaces rather than presenting the method's derivation or making new benchmark-performance claims.

Software design

The public API separates configuration, evaluation, modeling, candidate selection, and orchestration. `RuntimeConfig` controls tensor precision and device placement without requiring source edits, and typed `QPOTSConfig` objects validate optimization settings. The extensible `Function` interface handles input conversion and output-shape validation while delegating objective evaluation to a callable or subclass hook; `EvaluationResult` represents objectives and optional constraints explicitly, and a registry supplies the built-in benchmarks. `ModelObject`

is responsible for independent or multitask Gaussian-process fitting, while Acquisition is responsible for candidate and optional task selection.

QPOTSRunner composes those components into complete `run()` and incremental `step()` workflows. It owns the optimization loop, works in physical design coordinates at the user boundary, and exposes normalized coordinates in result objects where the surrogate workflow needs them. Model and acquisition factories, runtime configuration, and iteration callbacks are injected through the runner rather than fixed inside it. This keeps each component independently replaceable and allows new benchmark functions, models, acquisitions, and reporting callbacks to be added without modifying the runner's control flow.

The design makes two computational trade-offs explicit. Posterior-sample optimization adds stochastic evolutionary-search overhead, but avoids requiring a differentiable analytical acquisition and naturally generates a sampled Pareto set. The package therefore delegates sampled-path search to NSGA-II through pymoo and retains BoTorch and GPyTorch for probabilistic modeling instead of reimplementing either dependency. Coupled and decoupled workflows share this candidate-generation path; decoupled runs add multitask modeling and optional task selection while representing skipped outputs as missing observations. Figure 1 connects these components in a constrained two-objective run: evaluated designs update the surrogate, posterior-path optimization approximates a Pareto set, and the diversity rule selects the next batch.

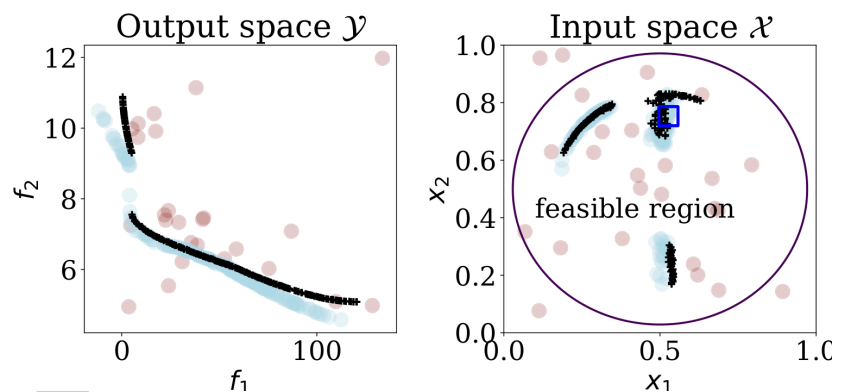


Figure 1: Illustration of qPOTS on a constrained two-objective problem. Shown are the initial training data, the Pareto front approximation, and the batch of candidates proposed by the acquisition strategy.

Figure 2 illustrates the additional information available to a decoupled run: task-specific observations produce a multitask posterior whose cross-task correlation can guide output selection.

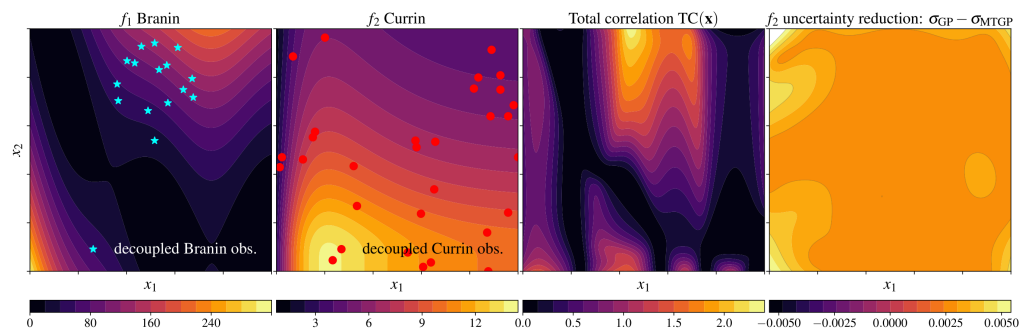


Figure 2: The qPOTS-Decoupled multitask update uses posterior task correlation to decide when decoupled evaluations are informative. The Branin-Currin example illustrates task-specific observations, total-correlation evolution, and uncertainty reduction from multitask information sharing.

Research impact statement

qPOTS is the reference implementation for the AISTATS 2025 paper introducing Pareto optimal Thompson sampling for batch multiobjective Bayesian optimization (Renganathan & Carlson, 2025). The method was evaluated on eight synthetic and real-world benchmarks against qEHVI, qNEHVI, qLogNEHVI, ParEGO, PESMO, MESMO, JESMO, TS-EMO, and Sobol baselines, demonstrating competitive or superior hypervolume improvements with substantially lower computational cost per iteration compared to Monte Carlo acquisition methods. The package has also supported an applied aerodynamic design study of the NASA Common Research Model, where qPOTS was deployed for 24-dimensional, two-objective aerodynamic design optimization and released with open-source reproducibility materials (Carlson & Renganathan, 2026).

The current software release also supports correlated objective and constraint tasks with partial observations. The repository provides tests and worked examples for that workflow, hosted documentation at ReadTheDocs, and PyPI and source installation paths. The companion methods manuscript reports the algorithmic derivation and benchmark analysis (Renganathan et al., 2026); those results are not repeated as claims of this software paper.

Independent evaluation is beginning to emerge: SPREAD includes qPOTS in a numerical comparison of multiobjective Bayesian optimization methods (Hotejni & Peitz, 2025). Other publications cite qPOTS while discussing generative molecular design, sustainable process systems, class-imbalance learning, and many-objective optimization (Jiang et al., 2026; Kudva et al., 2026; Muthyala et al., 2026; Wang et al., 2025). We treat those citations as evidence of both awareness of qPOTS and as a state-of-the-art benchmark in the multiobjective Bayesian optimization community.

AI usage disclosure

Anthropic Claude was used to assist with refactoring the code and structuring the Readme.md before submission. Generative AI was not used to design the qPOTS or qPOTS-Decoupled algorithms, to implement the software, or to generate any benchmark results. The authors are responsible for all technical claims in this manuscript; claims should be verified against the source code, hosted documentation, the AISTATS qPOTS paper (Renganathan & Carlson, 2025), and the companion decoupled-evaluation manuscript (Renganathan et al., 2026).

Acknowledgements

The authors acknowledge the Penn State Institute for Computational and Data Sciences for access to computational research infrastructure through the Roar Core Facility.

References

- Álvarez, M. A., Rosasco, L., & Lawrence, N. D. (2012). Kernels for vector-valued functions: A review. *Foundations and Trends in Machine Learning*, 4(3), 195–266. <https://doi.org/10.1561/22000000036>
- Ament, S., Daulton, S., Eriksson, D., Balandat, M., & Bakshy, E. (2023). Unexpected improvements to expected improvement for bayesian optimization. *Advances in Neural Information Processing Systems*, 36, 20577–20612. <https://doi.org/10.52202/075280-0904>
- Balandat, M., Karrer, B., Jiang, D., Daulton, S., Letham, B., Wilson, A. G., & Bakshy, E. (2020). BoTorch: A framework for efficient monte-carlo bayesian optimization. *Advances in Neural Information Processing Systems*, 33, 21524–21538. <https://proceedings.neurips.cc/paper/2020/hash/f5b1b89d98b7286673128a5fb112cb9a-Abstract.html>
- Belakaria, S., Deshwal, A., & Doppa, J. R. (2019). Max-value entropy search for multi-objective bayesian optimization. *Advances in Neural Information Processing Systems*, 32. https://proceedings.neurips.cc/paper_files/paper/2019/hash/82edc5c9e21035674d481640448049f3-Abstract.html
- Blank, J., & Deb, K. (2020). pymoo: Multi-objective optimization in python. *IEEE Access*, 8, 89497–89509. <https://doi.org/10.1109/ACCESS.2020.2990567>
- Bonilla, E. V., Chai, K. M. A., & Williams, C. K. I. (2007). Multi-task gaussian process prediction. *Advances in Neural Information Processing Systems*, 20. <https://proceedings.neurips.cc/paper/2007/hash/66368270ffd51418ec58bd793f2d9b1b-Abstract.html>
- Carlson, K., & Renganathan, A. (2026). Multiobjective aerodynamic design optimization of the NASA common research model. *Aerospace Science and Technology*, 168, 111120. <https://doi.org/10.1016/j.ast.2025.111120>
- Daulton, S., Balandat, M., & Bakshy, E. (2020). Differentiable expected hypervolume improvement for parallel multi-objective bayesian optimization. *Advances in Neural Information Processing Systems*, 33, 9851–9864. <https://proceedings.neurips.cc/paper/2020/hash/6fec24eac8f18ed793f5eaa3dd7977c-Abstract.html>
- Daulton, S., Balandat, M., & Bakshy, E. (2023). Hypervolume knowledge gradient: A lookahead approach for multi-objective bayesian optimization with partial information. *Proceedings of the 40th International Conference on Machine Learning*, 202, 7167–7204. <https://proceedings.mlr.press/v202/daulton23a.html>
- Deb, K., Pratap, A., Agarwal, S., & Meyarivan, T. (2002). A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Transactions on Evolutionary Computation*, 6(2), 182–197. <https://doi.org/10.1109/4235.996017>
- Gardner, J., Pleiss, G., Weinberger, K. Q., Bindel, D., & Wilson, A. G. (2018). GPyTorch: Blackbox matrix-matrix gaussian process inference with GPU acceleration. *Advances in Neural Information Processing Systems*, 31, 7576–7586. https://proceedings.neurips.cc/paper_files/paper/2018/hash/27e8e17134dd7083b050476733207ea1-Abstract.html
- Garrido-Merchán, E. C., & Hernández-Lobato, D. (2019). Predictive entropy search for multi-objective bayesian optimization with constraints. *Neurocomputing*, 361, 50–68. <https://doi.org/10.1016/j.neucom.2019.05.025>
- Hotegni, S. S., & Peitz, S. (2025). SPREAD: Sampling-based pareto front refinement via

- 183 efficient adaptive diffusion. *arXiv Preprint arXiv:2509.21058*. [https://doi.org/10.48550/](https://doi.org/10.48550/arXiv.2509.21058)
184 [arXiv.2509.21058](https://doi.org/10.48550/arXiv.2509.21058)
- 185 Jiang, C., Huang, J., & Li, M. (2026). Do we really need to approach the entire pareto
186 front in many-objective bayesian optimisation? *arXiv Preprint arXiv:2604.09417*. [https://](https://doi.org/10.48550/arXiv.2604.09417)
187 doi.org/10.48550/arXiv.2604.09417
- 188 Jones, D. R., Schonlau, M., & Welch, W. J. (1998). Efficient global optimization of expensive
189 black-box functions. *Journal of Global Optimization*, 13(4), 455–492. [https://doi.org/10.](https://doi.org/10.1023/A:1008306431147)
190 [1023/A:1008306431147](https://doi.org/10.1023/A:1008306431147)
- 191 Kudva, A., Tang, W.-T., & Paulson, J. A. (2026). Multi-objective bayesian optimization for
192 networked black-box systems: A path to greener profits and smarter designs. *Optimization*
193 *of Sustainable Process Systems: Multiscale Models and Uncertainties*, 63–89. [https://](https://doi.org/10.1002/9781394205646.ch3)
194 doi.org/10.1002/9781394205646.ch3
- 195 Muthyala, M. R., Sorourifar, F., Tan, T., Peng, Y., & Paulson, J. A. (2026). Generative
196 multiobjective bayesian optimization with scalable batch evaluations for sample-efficient
197 de novo molecular design. *Industrial & Engineering Chemistry Research*, 65(1), 628–642.
198 <https://doi.org/10.1021/acs.iecr.5c03166>
- 199 Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z.,
200 Gimelshein, N., Antiga, L., Desmaison, A., Kopf, A., Yang, E., DeVito, Z., Raison, M.,
201 Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., ... Chintala, S. (2019). PyTorch: An
202 imperative style, high-performance deep learning library. *Advances in Neural Information*
203 *Processing Systems*, 32, 8024–8035. [https://proceedings.neurips.cc/paper/2019/hash/](https://proceedings.neurips.cc/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html)
204 [bdbca288fee7f92f2bfa9f7012727740-Abstract.html](https://proceedings.neurips.cc/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html)
- 205 Rasmussen, C. E., & Williams, C. K. I. (2006). *Gaussian processes for machine learning*. MIT
206 Press. <https://gaussianprocess.org/gpml/>
- 207 Renganathan, A., & Carlson, K. (2025). qPOTS: Efficient batch multiobjective bayesian
208 optimization via pareto optimal thompson sampling. In Y. Li, S. Mandt, S. Agrawal, &
209 E. Khan (Eds.), *Proceedings of the 28th international conference on artificial intelligence*
210 *and statistics* (Vol. 258, pp. 4051–4059). PMLR. [https://proceedings.mlr.press/v258/](https://proceedings.mlr.press/v258/renganathan25a.html)
211 [renganathan25a.html](https://proceedings.mlr.press/v258/renganathan25a.html)
- 212 Renganathan, A., Carlson, K., & Bachman, P. E. (2026). *qPOTS-DOE: Batch pareto optimal*
213 *thompson sampling with decoupled oracle evaluations for constrained multiobjective bayesian*
214 *optimization*.
- 215 Shahriari, B., Swersky, K., Wang, Z., Adams, R. P., & Freitas, N. de. (2016). Taking the
216 human out of the loop: A review of bayesian optimization. *Proceedings of the IEEE*,
217 104(1), 148–175. <https://doi.org/10.1109/JPROC.2015.2494218>
- 218 Thompson, W. R. (1933). On the likelihood that one unknown probability exceeds another in
219 view of the evidence of two samples. *Biometrika*, 25(3-4), 285–294. [https://doi.org/10.](https://doi.org/10.1093/biomet/25.3-4.285)
220 [1093/biomet/25.3-4.285](https://doi.org/10.1093/biomet/25.3-4.285)
- 221 Tu, B., Gandy, A., Kantas, N., & Shafei, B. (2022). Joint entropy search for multi-
222 objective bayesian optimization. *Advances in Neural Information Processing*
223 *Systems*, 35, 9922–9938. [https://proceedings.neurips.cc/paper_files/paper/2022/](https://proceedings.neurips.cc/paper_files/paper/2022/hash/4086fe59dc3584708468fba0e459f6a7-Abstract-Conference.html)
224 [hash/4086fe59dc3584708468fba0e459f6a7-Abstract-Conference.html](https://proceedings.neurips.cc/paper_files/paper/2022/hash/4086fe59dc3584708468fba0e459f6a7-Abstract-Conference.html)
- 225 Wang, Z., Wang, S., Ernst, D., & Xiao, C. (2025). Automated class imbalance learning via
226 few-shot multi-objective bayesian optimization with deep kernel gaussian processes. *IEEE*
227 *Access*. <https://doi.org/10.1109/access.2025.3591034>